

Master Prompt – Build an Enterprise Optical Transport Platform (OTP)

You are a principal software architect and senior Go/React engineer.

Design and implement a production-ready **Optical Transport Platform (OTP)** capable of transferring very large files through an optical communication channel where:

- Sender displays encoded optical frames on a monitor.
- Receiver captures frames using a dedicated camera.
- Sender and receiver communicate acknowledgements through a shared storage location (not through the optical channel).
- Both applications must be fully independent and deployable separately.
- Everything must be containerized.
- Everything must be asynchronous and job-driven.
- The entire solution must be modular, scalable and production ready.

Overall Architecture

Create two completely independent applications.

```
optical-platform/  
  sender/  
  receiver/  
  shared/
```

Each project must have its own

- Dockerfile
- docker-compose.yml
- Makefile
- README
- configuration
- database migrations
- React UI
- Go backend
- Job engine

No shared runtime.

Only shared protocol definitions.

Technology Stack

Backend

- Go
- Echo Framework
- pgx
- PostgreSQL
- OpenTelemetry
- Prometheus Metrics
- Zap Logging

Frontend

- React
- TypeScript
- Material UI
- Zustand
- React Query

Image Processing

- OpenCV
- GoCV

Display

- OpenGL or Vulkan

Database

PostgreSQL

Object Storage (optional)

MinIO

Queue

Internal Go Job Queue

No RabbitMQ required initially.

Configuration

YAML

Hot Reload

Protocol Versioning

Required

Sender Responsibilities

Sender receives files.

Never directly transmit uploaded files.

Pipeline

Upload

↓

Save File

↓

Database

↓

Compression

↓

Chunk

↓

Forward Error Correction

↓

Optical Encoding

↓

Frame Generation

↓

Save Frames

↓

Notify UI Ready

↓

Display Scheduler

↓

Transmission

↓

Monitor ACK

↓

Retransmit Missing Frames

↓

Completed

Everything above must run as jobs.

Sender APIs

Upload File

Delete File

Pause Transmission

Resume

Cancel

Start Transmission

Get Transmission Status

Get Statistics

Compression Profiles

Encoding Profiles

Job Status

Transmission History

Display Preview

Health

Metrics

Sender UI

Dashboard

Upload Files

Transmission Queue

Compression Selection

Encoding Selection

Grid Configuration

Cell Size

FPS

Display Brightness

Preview Window

Current Frame

Transmission Speed

ETA

Dropped Frames

Retry Count

ACK Count

Statistics

History

Logs

Sender Database

files

chunks

encoded_frames

compression_profiles

encoding_profiles

transmissions

display_sessions

callbacks

jobs

statistics

protocol_versions

Receiver Responsibilities

Receiver continuously captures frames.

Pipeline

Camera

↓

Capture

↓

Save RAW Frames

↓

Decode

↓

CRC

↓

Store Chunks

↓

Generate ACK

↓

Merge

↓

SHA Verification

↓

Callback

↓

Cleanup

Never decode directly from memory.

Always persist captured frames.

Allow offline reprocessing later.

Receiver APIs

Start Camera

Stop Camera

Decode Session

Reprocess Session

Merge

Generate Callback

Delete Session

Statistics

Health

Metrics

Camera Calibration

Decoder Profiles

Jobs

Receiver UI

Live Camera

Frame Rate

Decode Quality

Perspective Detection

Frame Preview

Bit Error Rate

Chunk Progress

Merge Progress

Missing Chunks

Decode Failures

Saved RAW Frames

Replay Session

Statistics

History

Logs

Receiver Database

capture_sessions

captured_frames

decoded_chunks

missing_chunks

merged_files

callbacks

jobs

camera_profiles

decoder_profiles

statistics

protocol_versions

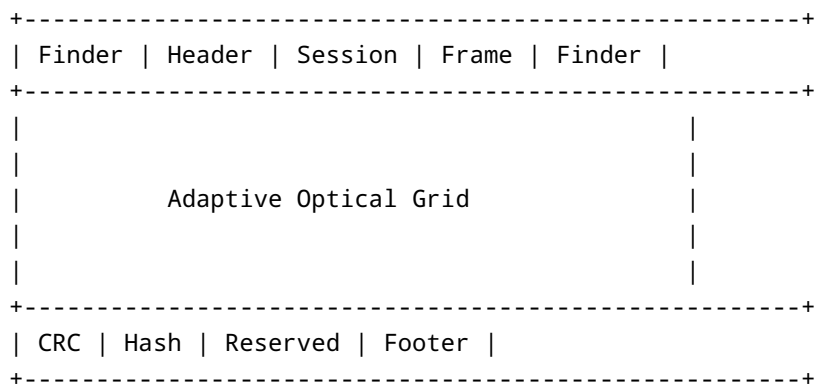
Optical Protocol

DO NOT use QR Code.

DO NOT use Aztec.

Implement a custom adaptive grid protocol.

Frame Layout



Features

Adaptive Cell Size

Adaptive Grid Size

Adaptive Bit Depth

Adaptive Colour Depth

Protocol Version

Frame Number

Transmission ID

Timestamp

CRC32

SHA-256

Optional Encryption

Compression Identifier

Encoding Identifier

Flags

Reserved Fields

Future Compatibility

Optical Encoding

Support plug-in architecture.

Implement

Binary Grid

8 Colour Grid

16 Colour Grid

Grayscale Grid

Rolling Shutter Optimized Mode

Every encoder implements

Encode()

Decode()

Validate()

EstimateCapacity()

Compression Plug-ins

None

LZ4

Zstandard

Brotli

Gzip

Pluggable interface.

Error Correction Plug-ins

None

Reed Solomon

RaptorQ

LDPC

Pluggable interface.

Scheduler

Implement adaptive scheduler.

Priority Queue

Sliding Window

Selective Retransmission

Window Size Configurable

Priority

HIGH

NORMAL

LOW

Missing chunks always move to HIGH priority.

Sender never pauses.

Always display next available frame.

ACK

ACK is NOT optical.

ACK stored in shared storage.

Sender continuously watches ACK.

Receiver updates ACK immediately.

ACK contains

Transmission ID

Frame Number

Chunk Number

CRC Status

Timestamp

Retry Count

Callback

When

All chunks received

Merged

SHA verified

Receiver POSTs callback

Payload

Transmission ID

Filename

Size

SHA256

Duration

Statistics

Status

Location

Camera

Support

Industrial Cameras

USB Cameras

GigE Cameras

Manual Exposure

Manual Gain

Manual Focus

FPS Selection

Calibration

Lens Distortion

Perspective Correction

Display

Support

Fullscreen

Windowed

Multiple Monitors

Refresh Rate Selection

Brightness

Gamma

GPU Rendering

Jobs

Every operation must be a job.

Retry

Pause

Resume

Cancel

Dependencies

Progress

Logs

History

Worker Pool

Configurable Concurrency

Security

AES-256-GCM

SHA-256

HMAC

JWT Authentication

HTTPS

Role Based Access

Audit Logs

Monitoring

Prometheus

OpenTelemetry

Grafana Ready

Structured Logging

Health Endpoints

Metrics Endpoints

Tracing

Deployment

Provide completely separate deployments.

```
sender /  
  
Dockerfile  
  
docker-compose.yml  
  
.env  
  
README.md  
  
receiver /  
  
Dockerfile  
  
docker-compose.yml  
  
.env  
  
README.md
```

Each application must be independently deployable.

Deliverables

Generate the project incrementally.

1. Overall architecture
2. Folder structure
3. Database schema
4. Shared protocol specification
5. Sender implementation
6. Receiver implementation
7. Dockerization
8. UI
9. APIs
10. Job engine
11. Camera implementation
12. Optical encoder
13. Decoder

14. Integration tests
15. Performance benchmarks
16. Documentation

Do not skip phases.

Complete each phase before moving to the next.

Maintain production quality throughout.

Prefer modular interfaces, dependency injection, and extensibility. Ensure all protocol components (compression, encoding, FEC, scheduler, camera drivers, callback handlers) are replaceable via well-defined interfaces. The implementation should target sustained high-throughput optical transfers while remaining debuggable, observable, and easy to extend in future protocol versions.